Основи на Java EE
Серлвети и филтри

Веб програмирање

Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Сервлет контејнер и сервлети

- Јава платформи – JVM + колекција од API за развивање апликации
  - Java Standard Edition (JSE)
    - Јава библиотеки (Јава типови и класи, вмрежување, безбедност, пристап до бази на податоци, ...)
  - Java Enterprise Edition (Java EE)
    - Првично J2EE, па преименувана во Java EE
    - Екстензија на JSE за креирање на скалабилни, повеќеслојни, безбедни и доверливи апликации кои се извршуваат на сервер
    - Воведува управувани компоненти како сервлети, филтри и опсервери од страна на сервлет контејнери

Сервлет

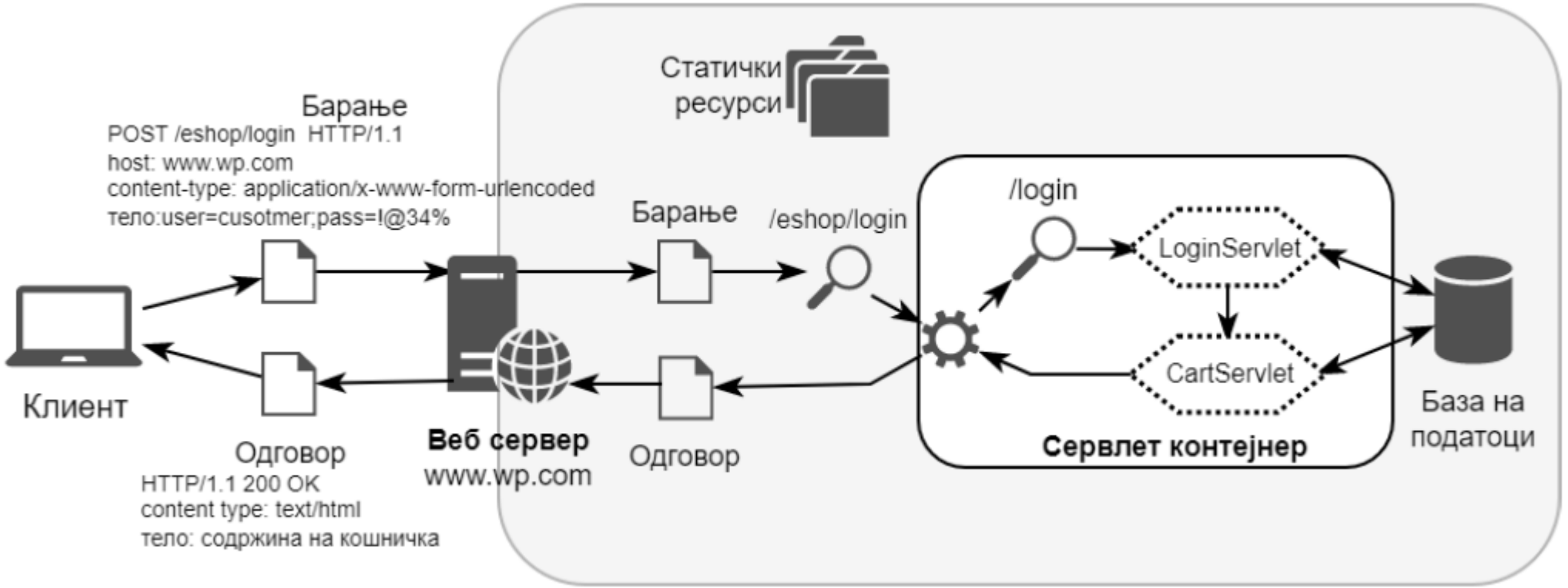
- Java класа која имплементира **бизнис логика** потребна за креирање динамички веб страници
  - Се повикува во зависност од побараната URL адреса
  - Нема main()
  - Контролиран од **сервлет контејнер**
  - Имплементира одредена функционалност на апликацијата
  - Една апликација е составена од колекција од сервлети

Сервлет контејнер

- Java процес кој управува со сервлети
  - Дел од Java базиран веб сервер (Apache Tomcat)
  - Ја олеснува комуникацијата помеѓу веб серверот и клиентот
    - Прима HTTP барање и го конвертира во инстанца од HttpServletRequest
    - Креира инстанца од HttpServletResponse
    - Ги предава објектите на еден од сервлетите
    - Креира HTTP одговор од инстанцата на HttpServletResponse
      - откако сервлетот ќе заврши со обработка
  - Управува со животниот циклус на сервлетите, филтрите и обсерверите
  - Нуди паралелизација
    - Обработка на повеќе барања одеднаш
  - Нуди заеднички простор за споделување на податоци

Сервлет контејнер и сервлети

- Комуникација помеѓу клиент, веб сервер и контејнер



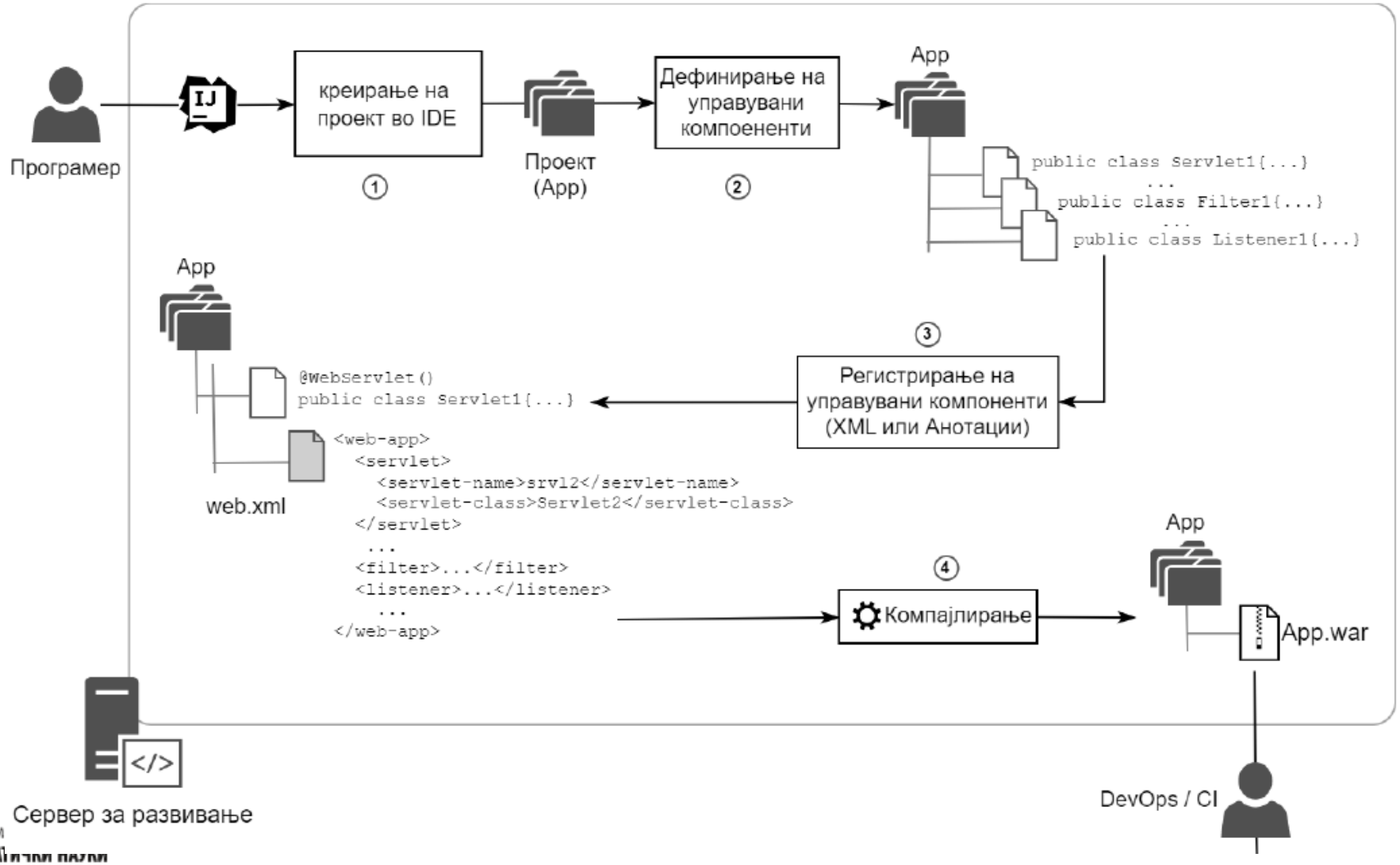
Филтер и опсервер

- Филтер (Filter) - компонента која пресретнува и модифицира барања и одговори пред да бидат предадени на сервлет
- Сервлет контејнерот може да повика повеќе филтри пред да го предаде барањето на сервлетот
- Опсервер – компонента која се користи за регистрирање за следење на одредени настани во рамките на сервлет контејнерот и дефиниција на соодветна акција
  - Креирање на барање
  - Креирање на корисничка сесија
  - ...

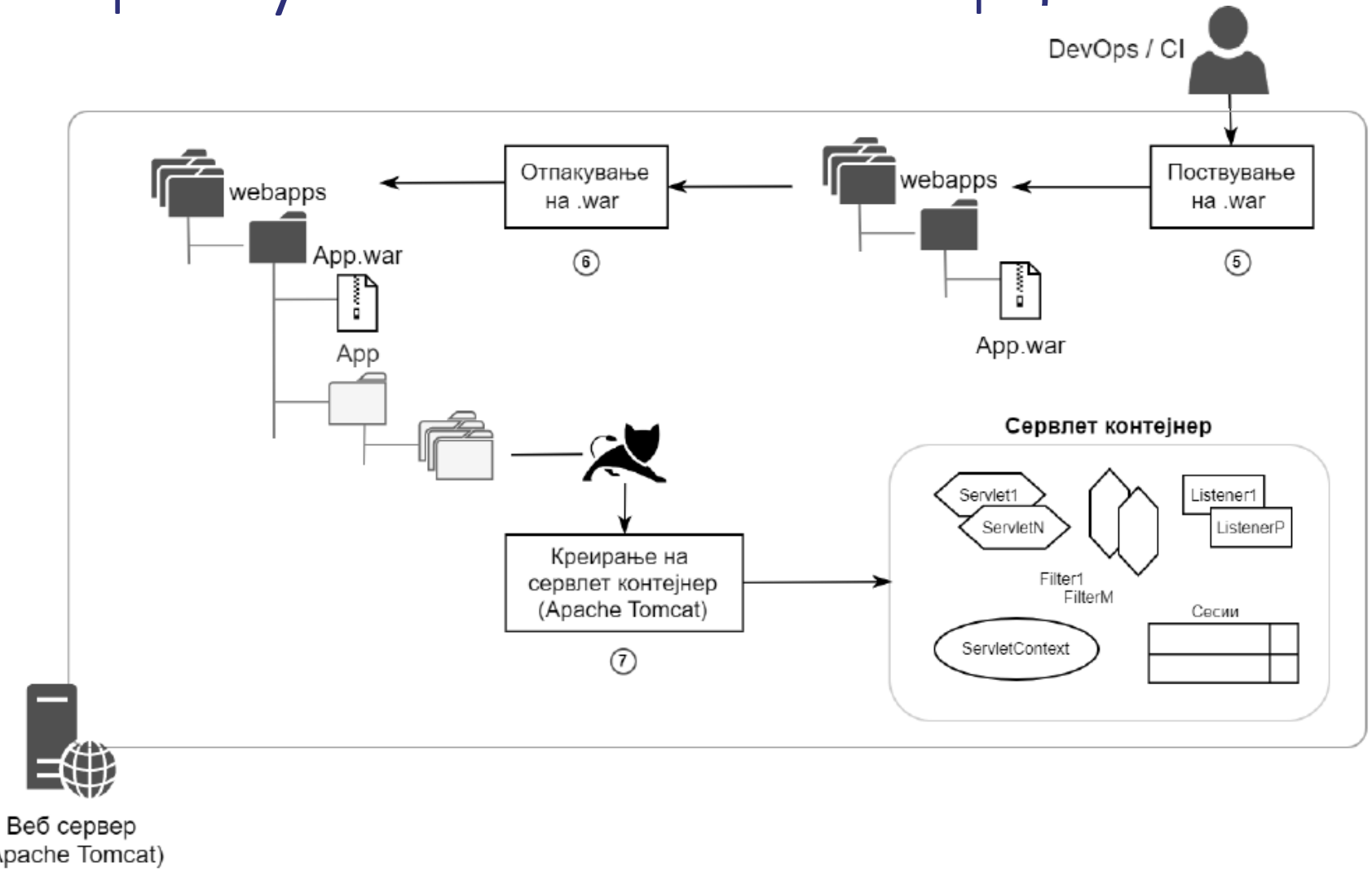
Регистрација на Java EE компоненти

- **Управуваните компоненти** од типот сервлети, филтри и опсервери се **класи** кои имплементираат **интерфејси** од Java EE API
- Сите **управувани** компоненти се **регистрираат** кај сервлет контејнерот преку
  - Дескриптор за поставување (Deployment descriptor)
    - **web.xml**
  - **Анотации** во самите компоненти
- Сите регистрирани компоненти преставуваат дел од апликацијата
- По имплементацијата, заедно со останатите ресурси се пакуваат во архива **.war**

Животен циклус на веб апликација



Животен циклус на веб апликација



Веб апликација со Spring Boot

- Содржи интегриран Apache Tomcat сервер
- Апликацијата најчесто се пакува во .jar
- Извршувањето на .jar го стартува и вграденият Apache Tomcat сервер
  - Апликацијата е самостојна и не е потребен дополнителен апликациски сервер
- Лесно создавање на продукциска средина
  - Фокус на развој на апликациите
  - Минимален напор за нивно поставување

Сервлет

- Дефиниција на генерички сервлет

```
1  import javax.servlet.*
2  public class FirstGenericServlet extends GenericServlet{
3
4      public void service(ServletRequest request, ServletResponse response) {
5          ...
6      }
7  }
```

- `GenericServlet` го имплементира интерфејсот `Servlet` со методи `init()`, `service()` и `destroy()`

Сервлет

- Дефиниција на HttpServlet (за HTTP комуникација)

```
public class FirstHttpServlet extends HttpServlet{
    doGet(HttpServletRequest request, HttpServletResponse response){
        ...
    }
    doPost(HttpServletRequest request, HttpServletResponse response){
        ...
    }
    ...
}
```

  - Постојат и `doPut()`, `doDelete()`, `doHead()`, `doOptions()` и `doTrace()`
  - Сите претходни методи се повикани од страна на `service()`

HttpServletRequest

- String getMethod()
- String getRemoteAddr()
- String getHeader(String header)
- Enumeration getHeaders(String header)
- Enumeration getHeaderNames()
- Enumeration getHeaders()
- String getContentType()
- int getContentLength()
- Cookie[] getCookies()
- String getParameter(String name)
- Enumeration getParameterNames()
- String[] getParameterValues(String name)
- ServletInputStream getInputStream()
- BufferedReader getReader()

HttpServletResponse

- void setHeader(String header, String value)String getRemoteAddr()
- String getHeader(String header)
- void addHeader(String header, String value)
- boolean containsHeader(String header)
- Enumeration<String> getHeaders(String header)
- Enumeration<String> getHeaderNames()
- void setStatus(int status)
- int getStatus(int status)
- void sendError(int status, String message)
- void addCookie(Cookie cookie)
- void setContentType()
- ServletOutputStream getOutputStream()
- PrintWriter getWriter()

Регистрација на сервлети - анотации

- Пример за единечно пресликување

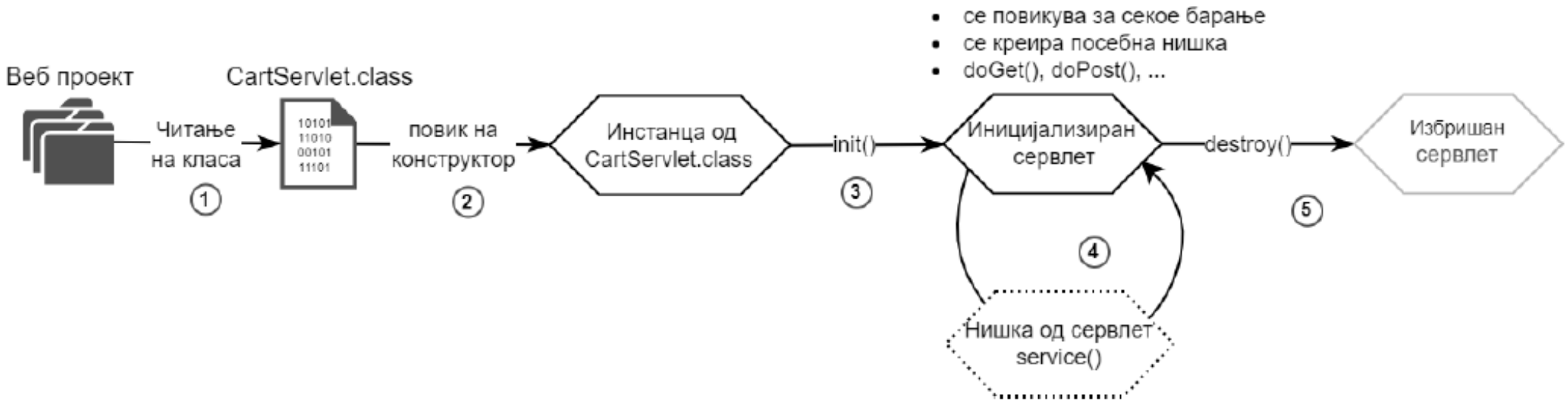
```
@WebServlet(name = "cart", value = "/show-cart")
public class ShoppingCart extends HttpServlet {
    ...
}
```

- Пример за повеќекратно пресликување
  - name може да се изостави (и во горниот пример)

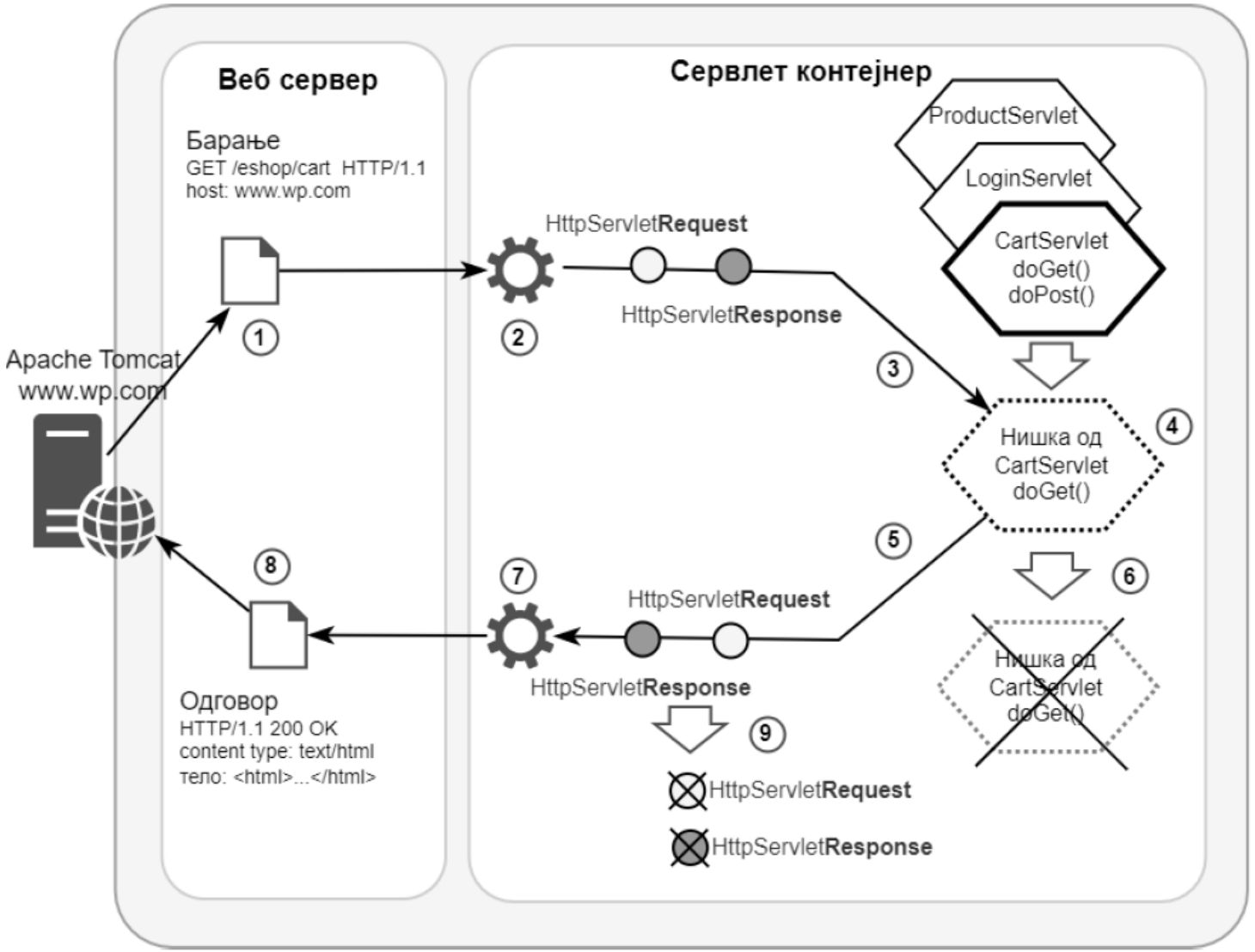
```
@WebServlet(ulrPatterns = {"/show-cart", "/cart"})
public class ShoppingCart extends HttpServlet {
    ...
}
```

- web.xml vs. @WebServlet?

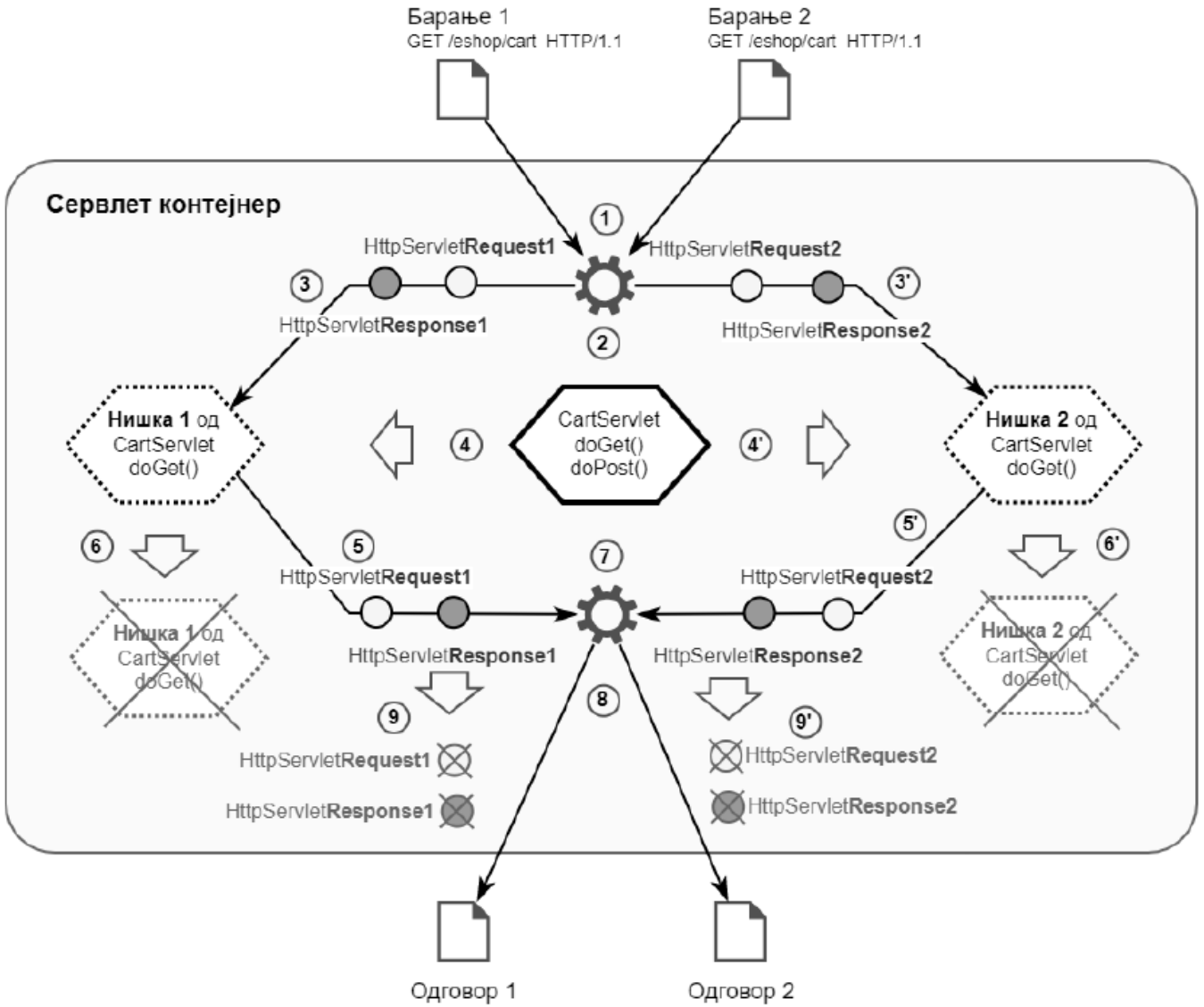
Животен циклус на сервлет



Обработка на барање од сервлет



Обработка на повеќе барања од сервлет



Пример

index.html

```
<html>
    <body>
        <a href="servlet">Create empty GET request</a><br>
        <a href="servlet?q=spring">Create GET request with query string</a>
            <form action="servlet" method="post">
                <h2>Insert a value</h2>
                <input type="text" name="input_data"/>
                <input type="submit" value="submit">
            </form>
    </body>
</html>
```



Пример

SimpleServlet.class

```
@WebServlet(name = "SimpleServlet", value = "/servlet")
public class SimpleServlet extends HttpServlet {
    public void init() {
        System.out.println("SimpleServlet is initialized!");
    }
    public void doGet(HttpServletRequest request, HttpServletResponse
     ↪  response) throws IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.print(getHeaders(request));

        out.println("<h5>" + "Message from doGet() method in SimpleSerlvet."
         ↪  + "</h5>");
        if ( request.getParameter("q")!=null)
            out.println("<h6>" + "Received data:" +
             ↪  request.getParameter("q") + "</h6>");
        else
            out.println("<h5>No data received!</h5>");
        out.println("</body></html>");
        int rnd = new Random().nextInt();
        response.addCookie(new Cookie("firstCookie", String.valueOf(rnd)));
        System.out.println("SimpleServlet doGet() invoked!");
    }
}
```

Пример

```
public void doPost(HttpServletRequest request, HttpServletResponse
 ↪  response)
        throws IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<html><body>");
    out.print(getHeaders(request));

    out.println("<h5>" + "Message from doPost in SimpleSerlvet." +
     ↪  "</h5>");
    out.println("<h6>Received data: " +
     ↪  request.getParameter("input_data") + "</h6>");
    out.println("</body></html>");
    System.out.println("SimpleServlet  doPost() invoked!");
}
public void destroy() {
    System.out.println("SimpleServlet is destroyed!");
}
 private String getHeaders(HttpServletRequest request){
    StringBuilder builder = new StringBuilder();
    Enumeration<String> headerNames = request.getHeaderNames();
    while(headerNames.hasMoreElements())
    {
        String header = headerNames.nextElement();
        builder.append(header+": "+request.getHeader(header)+"<br> ");
    }
    return  builder.toString();
}
```

Пример

- Резултат од прв линк

host: localhost:8080
connection: keep-alive
sec-ch-ua: " Not A;Brand";v="99", "Chromium";v="102", "Google Chrome";v="102"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
upgrade-insecure-requests: 1
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webexchange;v=b3;q=0.9
sec-fetch-site: same-origin
sec-fetch-mode: navigate
sec-fetch-user: ?1
sec-fetch-dest: document
referer: http://localhost:8080/ch2_handson_1_war_exploded/
accept-encoding: gzip, deflate, br
accept-language: en-GB,en;q=0.9

Message from doGet() method in SimpleSerlvet.

No data received!

- Конзола во IDE

```
SimpleServlet is initialized!
SimpleServlet doGet() invoked!
```

Пример

- Резултат од втор линк

host: localhost:8080
connection: keep-alive
sec-ch-ua: " Not A;Brand";v="99", "Chromium";v="102", "Google Chrome";v="102"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
upgrade-insecure-requests: 1
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webexchange;v=b3;q=0.9
sec-fetch-site: same-origin
sec-fetch-mode: navigate
sec-fetch-user: ?1
sec-fetch-dest: document
referer: http://localhost:8080/ch2_handson_1_war_exploded/
accept-encoding: gzip, deflate, br
cookie: firsCookie=-1193959466

Message from doGet() method in SimpleServlet.

Received data:spring

- Конзола во IDE

```
SimpleServlet doGet()invoked!
```

Пример

- Резултат од submit
  - после внесен текст post-data

host: localhost:8080
connection: keep-alive
content-length: 20
cache-control: max-age=0
sec-ch-ua: " Not A;Brand";v="99", "Chromium";v="102", "Google Chrome";v="102"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
upgrade-insecure-requests: 1
origin: http://localhost:8080
content-type: application/x-www-form-urlencoded
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/wel exchange;v=b3;q=0.9
sec-fetch-site: same-origin
sec-fetch-mode: navigate
sec-fetch-user: ?1
sec-fetch-dest: document
referer: http://localhost:8080/ch2_handson_1_war_exploded/
accept-encoding: gzip, deflate, br
accept-language: en-GB,en;q=0.9
cookie: firsCookie=-1193959466

Message from doPost in SimpleServlet.

Received data: post-data

- Конзола во IDE

```
SimpleServlet doPost()invoked!
```

  - Ако се прекине апликацијата

```
SimpleServlet is destroyed!
```

Во пракса

- За развивање на веб апликации во Spring се користат контролери кои, во зависност од барањето се повикани од страна на DispatcherServlet
  - Прима барања и ги препраќа до соодветен контролер
- Пример

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping
    public List<Product> getProducts() {
        return productService.findAll();
    }
}
```

Интеракција помеѓу сервлети

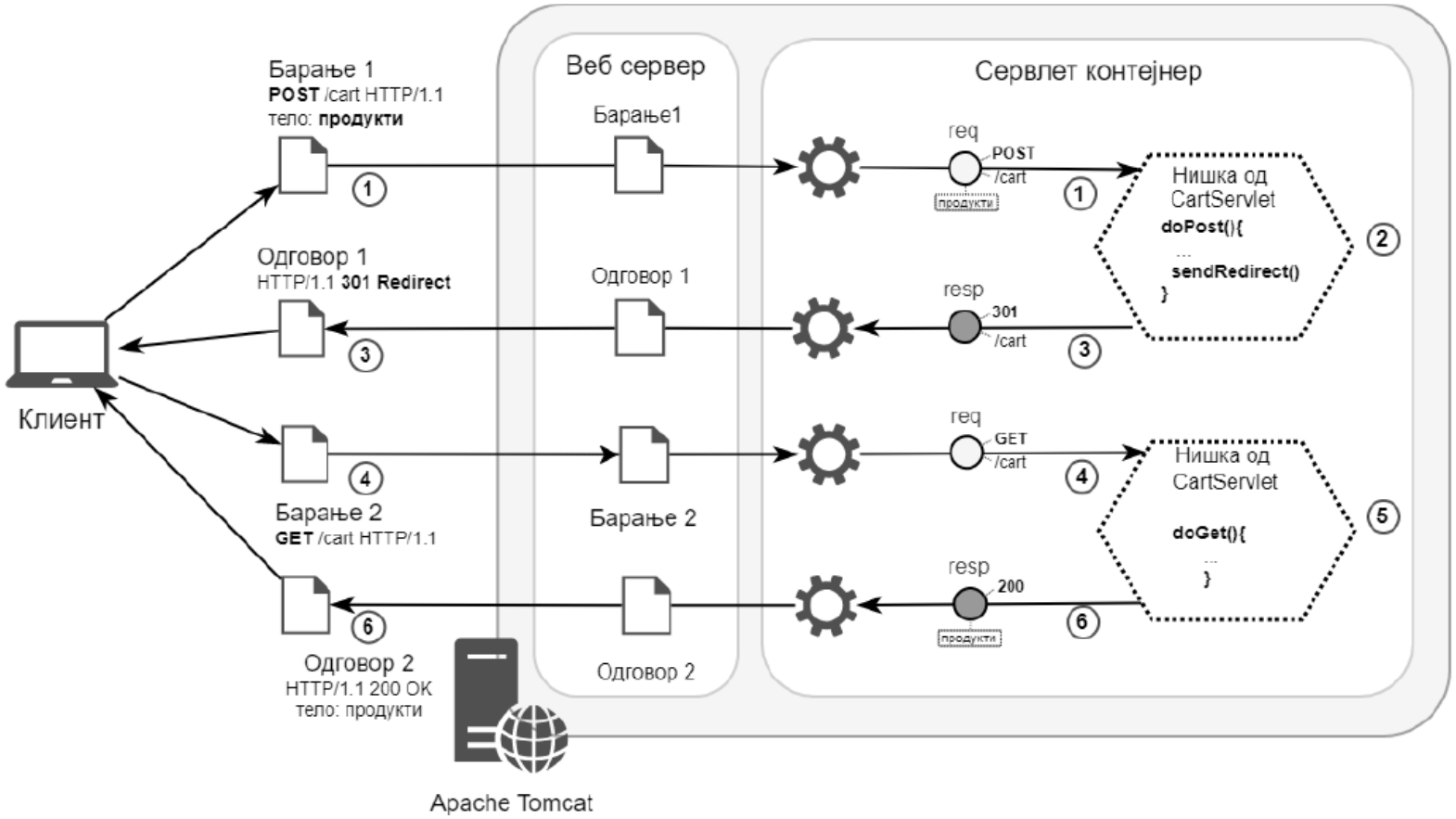
- Проследување на барање од еден сервлет до друг
  - SendRedirect
  - Forward
  - Include

Интеракција помеѓу сервлети

- SendRedirect

```
@WebServlet(name = "cartServlet", value = "/cart")
public class ShoppingCartServlet extends HttpServlet  {
    protected void doGet(HttpServletRequest request, HttpServletResponse
     ↪  response) throws IOException {
        // fill the response with html representing shopping cart
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse
     ↪  response) throws IOException {
        // retrieve data from request
        // update data into database
        response.sendRedirect("cart");
    }
}
```

Интеракција помеѓу сервлети



Интеракција помеѓу сервлети

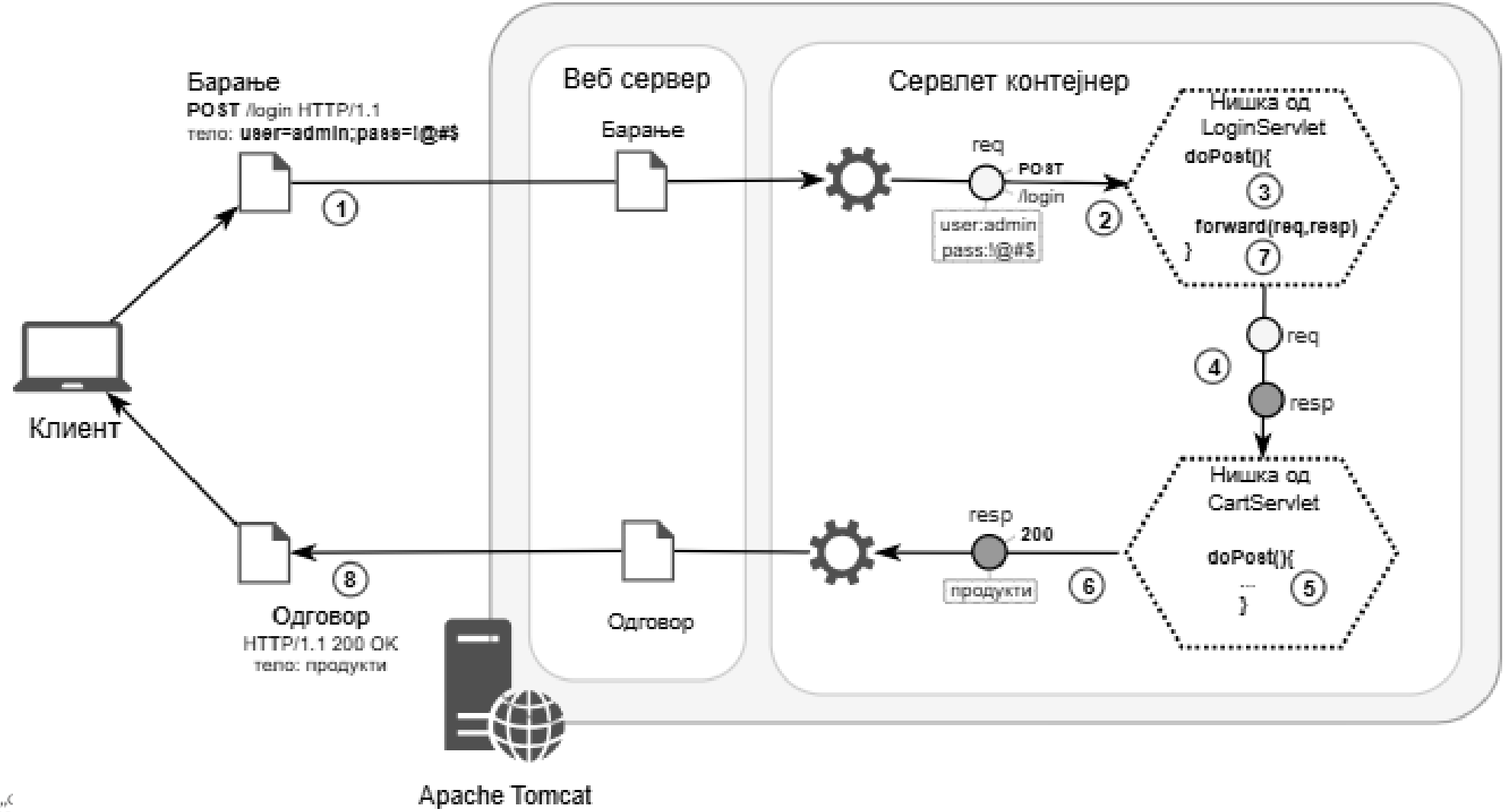
- Што ќе се случи ако корисникот направи Refresh во прелистувачот?
  - А) ќе се повика уште еднаш doPost (уште еднаш ќе се внесат продуктите)
  - Б) ќе се повика уште еднаш doGet (уште еднаш ќе се прикажат внесените продкути
- Точен одговор: Б
- sendRedirect се користи при бришење, додавање или уредување на ентитети - гарантира дека само еднаш ќе се извршат

Интеракција помеѓу сервлети

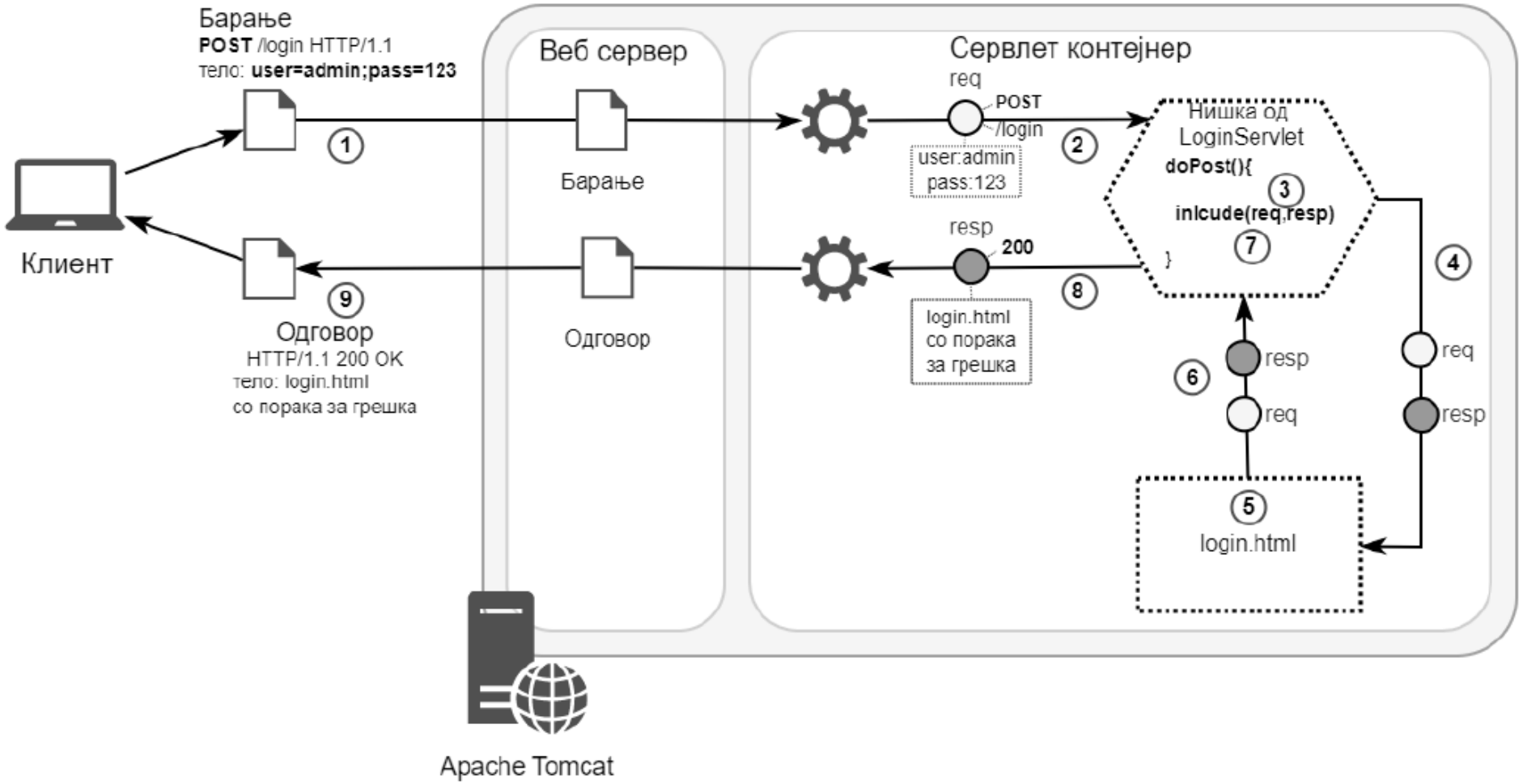
- Forward и include

```
@WebServlet(name = "login", value = "/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse
     ↪  response) throws IOException, ServletException {
        // retrieve username and password from request
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if (username.equals("admin") && password.equals("!@34")){
            RequestDispatcher dispatcher =
             ↪  request.getRequestDispatcher("cart");
            dispatcher.forward(request,response);
        }
        else {
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            out.print("Invalid user name or password!");
            RequestDispatcher
             ↪  dispatcher=request.getRequestDispatcher("/index.html");
            dispatcher.include(request, response);
        }
    }
}
```

Интеракција помеѓу сервлети



Интеракција помеѓу сервлети

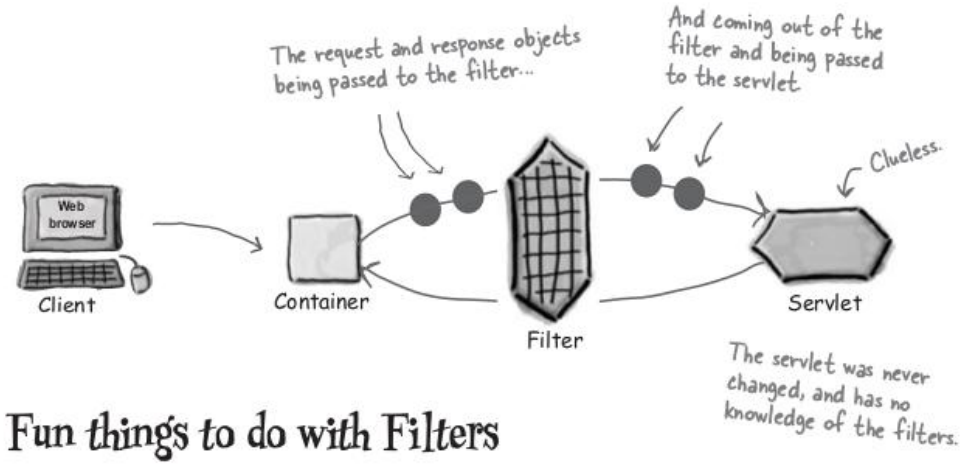


Филтри

Веб програмирање

Што се филтри

- Модуларни компоненти управувани од контејнерот
- Се вклучуваат со конфигурација
  - Слично како сервлетите
  - Независни од сервлетите



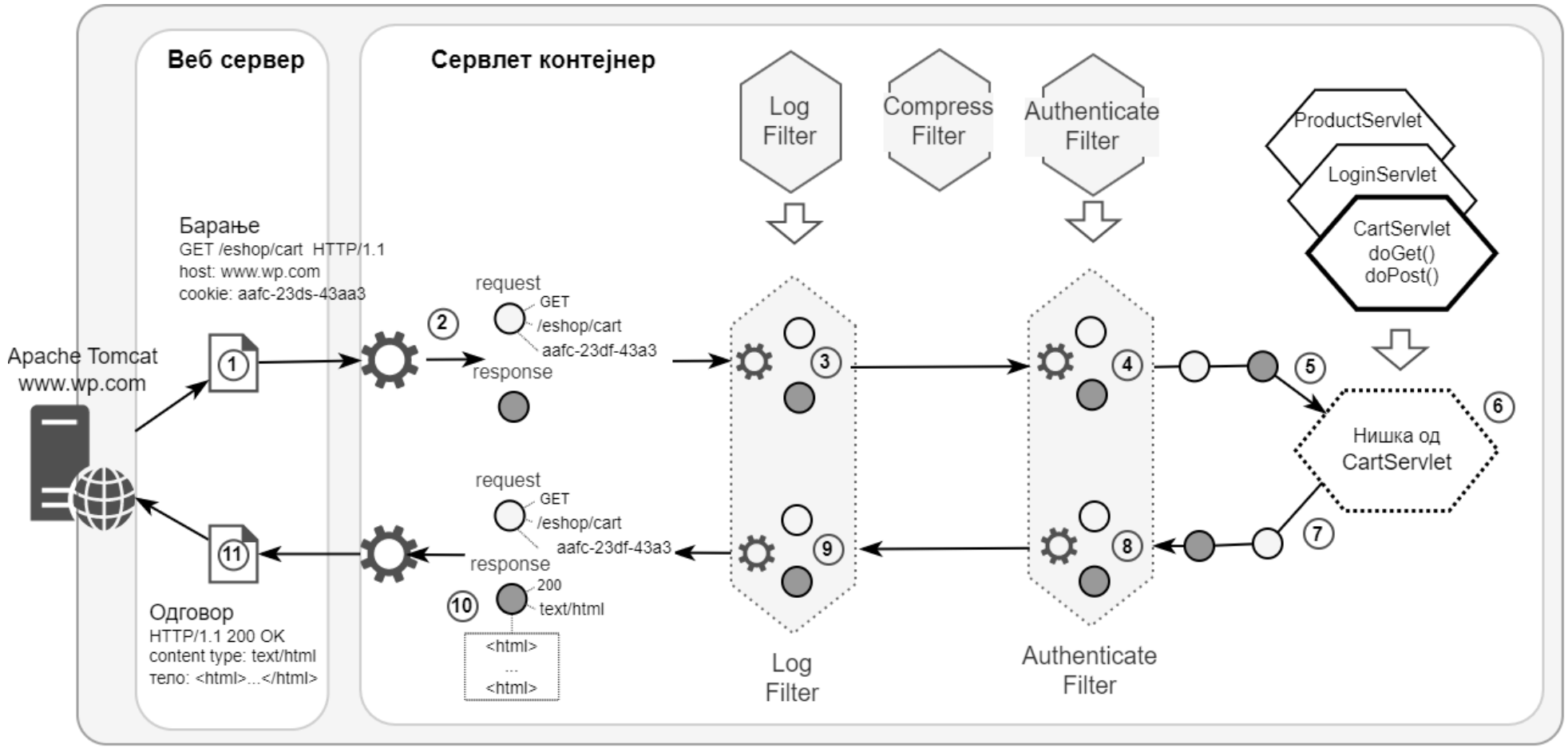
Зошто филтрите се од толку голема важност во пракса?

- Проверка на привилегии
  - Во секој сервлет посебно!!!
- Логирање на параметри и заглавја
  - Во секој сервлет посебно!!!
- Компресирање на одговорот
  - Во секој сервлет посебно!!!
- Генерирање на страна за недозволен пристап
  - Во секој сервлет посебно!!!
- Или посебен **филтер** за секоја **намена** ☺

Имплементација на филтри

```
public void doFilter(ServletRequest request, ServletResponse response,
    FilterChain chain) throws ServletException, IOException {
    //Preprocessing: the code above is executed before the objects are
        passed to the next filter in the chain in direction of activation
        towards the servlet
    chain.doFilter(request, response);
    //Postprocessing: the code below is executed when the objects are passed
        back in direction from the servlet towards the first filter
}
```

Претпроцесирање и постпроцесирање на барање и одговор со користење на филтри



Конфигурација на филтри и нивно пресликување во web.xml

- <filter-name> - името на претходно дефиниран филтер и
- <url-pattern> - URL адресата на барањето кое ќе го активира, или
- <servlet-name> - името на сервлетот за кој е наменето барањето. Овој елемент може да содржи листа од повеќе сервлети одвоени со запирка.
- <dispatcher>
  - REQUEST - за барања директно од клиентот
  - FORWARD - за барања кои потекнуваат од методот forward()
  - INCLUDE - за барања кои потекнуваат од методот include()
  - ERROR - за барања кои се проследуваат до конфигурирана страница за грешки (ако таква страна не е конфигурирана во web.xml, филтерот не се активира).

Конфигурација на филтри со анотација

```
@WebFilter(
    filterName = "timer",
    urlPatterns = "/*",
    dispatcherTypes = {DispatcherType.REQUEST, DispatcherType.FORWARD}
    initParams = {
        @WebInitParam(name = "timeout", value = "2000")
    })
public class TimerFilter implements Filter {
    ...
}
```

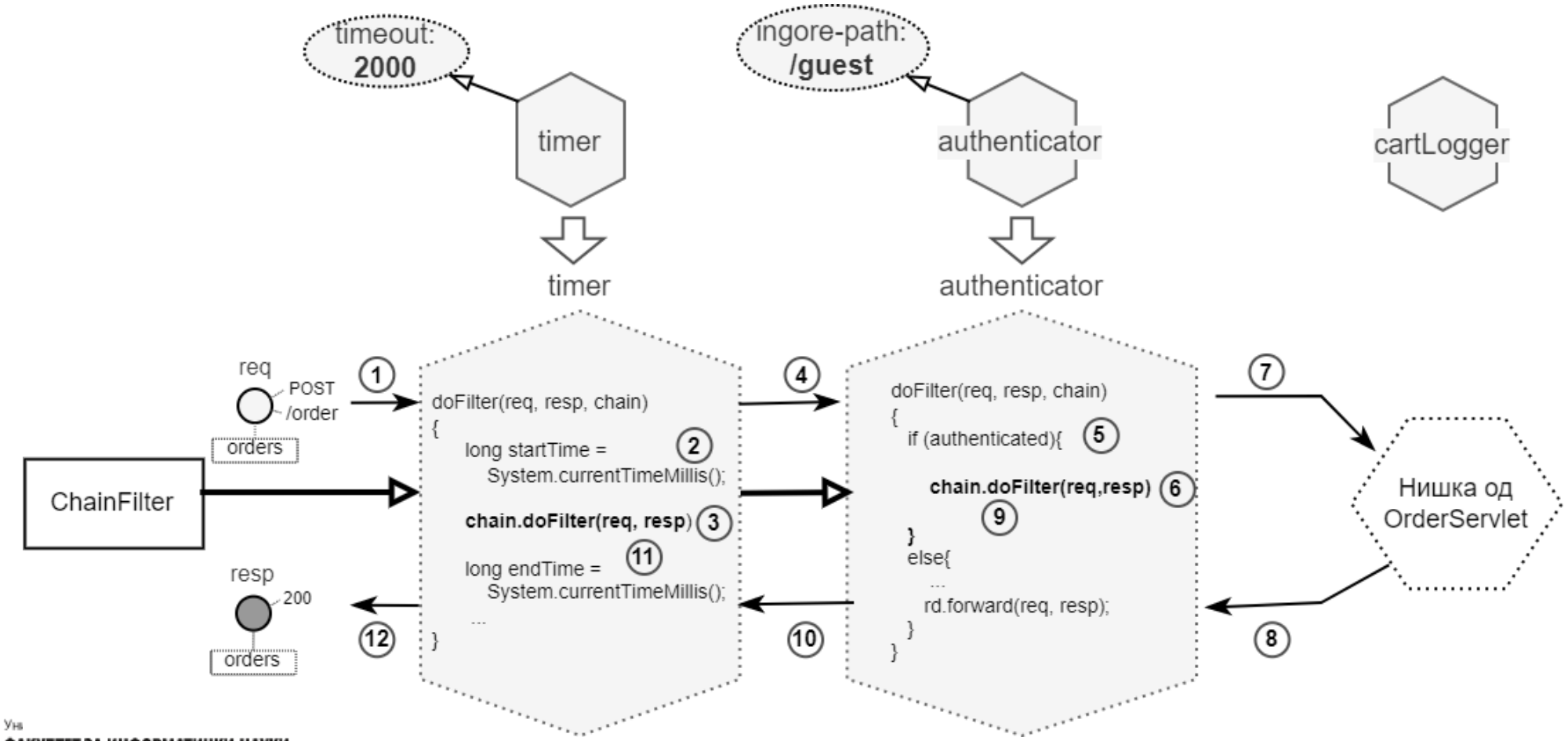
Пример 1. Имплементација на филтер за мерење на време на комплетна обработка на барање

```
1  public class TimerFilter implements Filter {
2    private ServletContext context;
3    private long timeout;
4    public void init(FilterConfig config) throws ServletException {
5      this.context = config.getServletContext();
6      this.timeout = (long)config.getParametar("timeout");
7    }
8    public void destroy() {}
9    public void doFilter(ServletRequest request, ServletResponse response,
         FilterChain chain) throws ServletException {
10     long startTime = System.currentTimeMillis();
11     chain.doFilter(request, response);
12     long endTime = System.currentTimeMillis();
13     long total endTime - startTime;
14     HttpServletRequest req = (HttpServletRequest) request;
15     String name = req.getRequestURI();
16     String comment = total > this.timeout ? " (too long!)": "";
17     this.context.log(name + " took " + total + " ms" + comment);
18   }
19 }
```

Пример 2. Имплементација на филтер за автентикација

```
1  public class AuthenticationFilter implements Filter {
2    privte String ignorePath;
3    public void init(FilterConfig config) throws ServletException {
4       this.ignorePath = config.getParametar("ignore-path");
5    }
6    public void destroy() {}
7    public void doFilter(ServletRequest request, ServletResponse response,
         FilterChain chain) throws ServletException {
8      HttpServletRequest req = (HttpServletRequest) request;
9      if (req.getRequestURI().startsWith(ingorePath) || req.getUserPrincipal()
           != null)
10         chain.doFilter(req,resp)
11     else{
12         RequestDispatcher rd=req.getRequestDispatcher("login.html");
13         rd.forward(req, resp);
14     }
15   }
16 }
```

Редослед на извршување на код од филтри при обработка на барање



Прашања?

